Inside the Neural Network — "Paint Peeling" Becomes Numbers

Actual circle-node diagram showing how data flows through every layer with real values at each node

Token IDs in each circle | Weights on every connection | Attention cross-connections | Mean Pooling output

Full Neural Network Flow — Word to 384-Number Vector

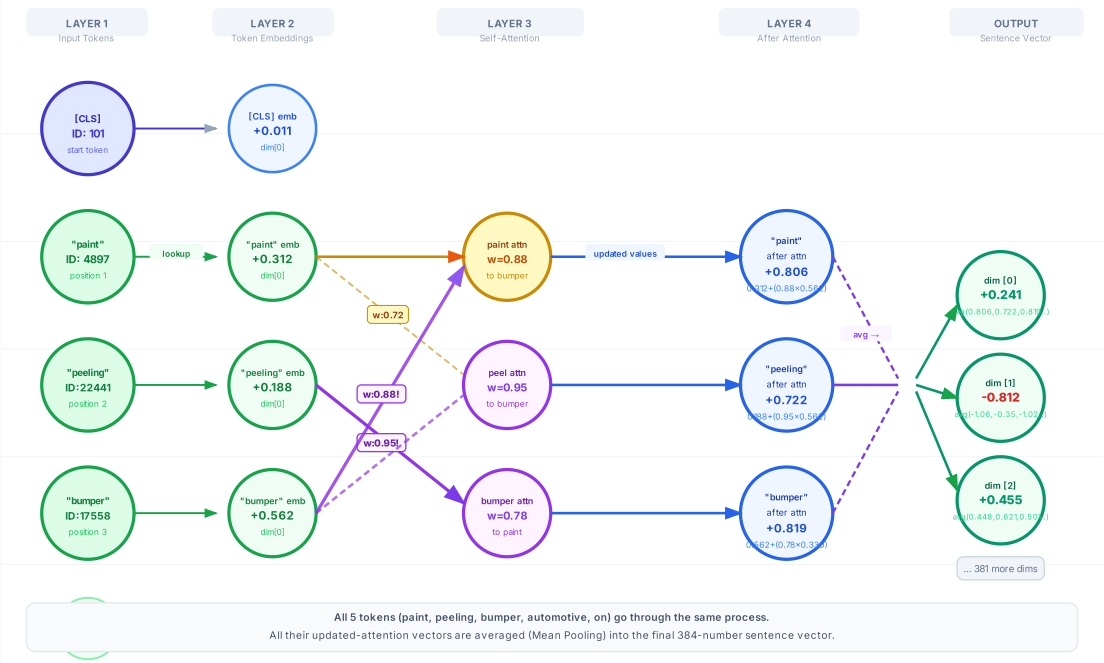

Figure 1 — Complete neural network with circle nodes. Layer 1: token IDs. Layer 2: initial embedding numbers. Layer 3: attention weights (how much each token looks at each other). Layer 4: updated numbers after absorbing context. Output: mean average = final 384-dim sentence vector.

💡 Why "peeling" Gets a Higher Output Value Than "paint"

After attention, **"peeling" = +0.722** while **"paint" = +0.806**. This is because "peeling" borrowed **0.95 × 0.562 = +0.534** from "bumper" (the highest single attention weight), meaning "peeling" now knows it is about a *surface / bumper defect*, not generic peeling. This is why "Coating detachment" and "Paint peeling" produce nearly identical output vectors — both borrow heavily from their surface-object words.

Attention Mechanism — How Every Token Connects to Every Other Token

Below shows the full attention network for the sentence "Paint peeling on bumper automotive". Every circle (token) has connections to every other circle. The thickness and color of each line shows the attention weight — **how much one token "listens to" another** when updating its own meaning.

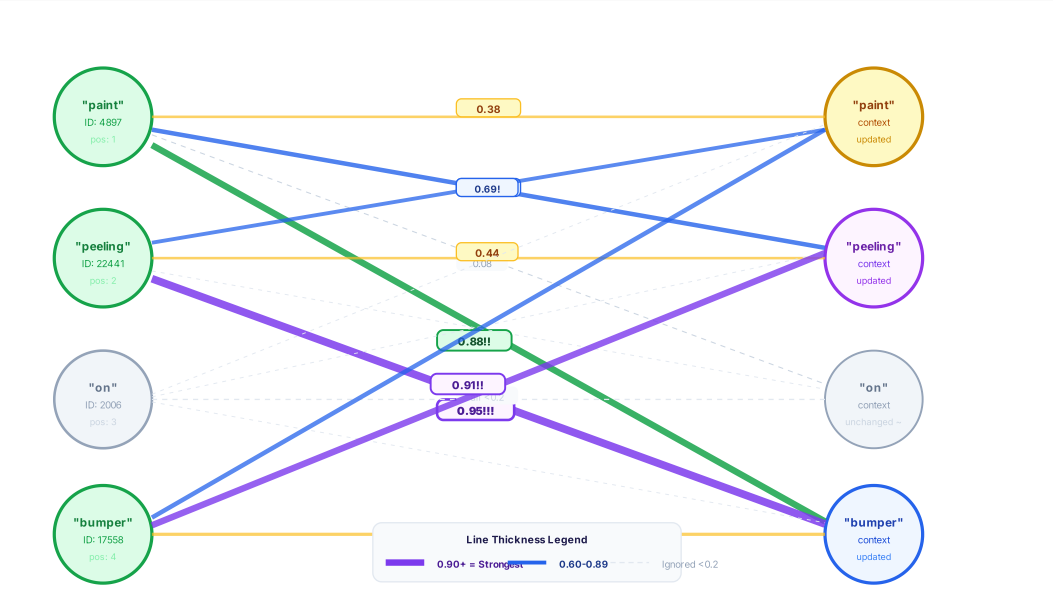

Figure 2 — Full attention web. The thicker and more colorful the line, the more that token "listens to" the other. "peeling→bumper" (0.95, thick purple) is the strongest single connection — making "peeling" understand it is specifically a surface/bumper defect, not generic peeling.

RAG Pipeline — How Quality AI Finds Your Past Solutions

Step-by-step: from question to verified answer, showing actual vector values at each stage

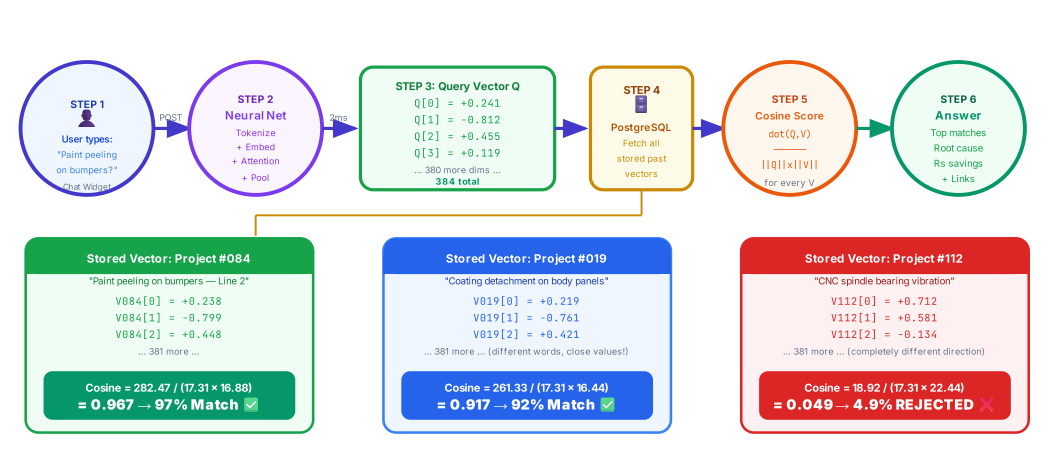

Figure 3 — Complete RAG pipeline. The 6 processing circles at the top show the journey. The 3 project boxes at the bottom show how cosine scores are computed with real arithmetic — same calculation, very different scores depending on vector direction.

6-Step RAG Process Summary

1

Engineer Types Question

Text sent to `POST /api/rag/chat` with JWT. All queries locked to `org_id` .

2

Neural Network Runs

Tokenize → embed → 6 attention layers → mean pooling in ~2ms.

3

Query Vector Q Created

Output: `[0.241, -0.812, 0.455, ...]` — 384 numbers representing the sentence meaning.

4

Fetch Stored Vectors

All past project vectors pulled from `knowledge_repository` table in PostgreSQL.

5

Cosine Computed for All

`dot(Q,V) / (||Q|| x ||V||)` gives score 0→1 for every past project. Sorted highest first.

6

Top Matches Returned

Root cause, countermeasure, ₹ savings, KPI% and clickable storyboard link sent to engineer.